Inheritance - 4 Types of Inheritance

Inheritance allows a new class **(child)** to automatically acquire the properties and methods of an existing class **(parent).**

Why it matters:

- **Code Reusability:** You don't have to rewrite code for common features; the child class automatically gets everything the parent has.
- **Hierarchy:** It creates a "is-a" relationship (e.g., an Electric Car is a Car).
- **Extensibility:** You can add specific features to the child class without modifying the parent class.

```
// Parent Class
class Car {
public:
    void drive() { cout << "Driving..."; }
};

// Child Class (Inherits from Car)
class ElectricCar : public Car {
public:
    void charge() { cout << "Charging battery..."; }
};

// Usage:
ElectricCar myTesla;
myTesla.drive();  // Inherited from Car
myTesla.charge(); // Unique to ElectricCar
```

Types of Inheritance

There are 4 Types of Inheritance.

- **Single** (One Parent, One Child)
- **Multilevel** (Grand Parent, Parent, Child)
- **Hierarchical** (More than one Child (Siblings))
- **Multiple** (More than one Parent) **(Diamond Problem)**

1. Single Inheritance: One child class inherits directly from exactly one parent class.

The Structure: Class A **->** Class B (A is the parent, B is the child).

```
class Car {};
class ElectricCar : public Car {}; // ElectricCar inherits from Car
```

2. **Multilevel Inheritance:** A class inherits from a child class, creating a chain of inheritance.

   The Structure: Class A -> Class B -> Class C

```
class Vehicle {};
class Car : public Vehicle {};
class SportsCar : public Car {}; // inherits from Car, which inherits from Vehicle
```

3. **Hierarchical Inheritance:** This happens when multiple distinct child classes inherit from one single parent class. It looks like a branching tree.

   The Structure:

   - Class A -> Class B
   - Class A -> Class C
   - Class A -> Class D

```
class Car {};
class Sedan : public Car {};
class SUV : public Car {}; // Both Sedan and SUV share the Car base
```

4. **Multiple Inheritance** occurs when a single child class inherits from **more than one** parent class.

   The Structure: Class A and Class B -> Class C.

```
class Engine { // Parent 1
public:
  void startEngine() { cout << "Engine started."; }
};

class Tech { // Parent 2
public:
  void connectGPS() { cout << "GPS connected."; }
};

// Child Class (Inherits from both)
class SmartCar : public Engine, public Tech {
  // Has access to both startEngine() and connectGPS()
};

// Usage:
SmartCar myCar;
myCar.startEngine();
myCar.connectGPS();
```

The Diamond Problem with Multiple Inheritance

If both **Parent A** and **Parent B** have a method with the same name (e.g., start()), the child class doesn't know which one to execute. This ambiguity is known as the **Diamond Problem**.

But with a grandparent for **Parent A** and **Parent B**, it gets even worse, known as the **Deadly Diamond of Death,** and causes two major problems.

1. The Ambiguity Problem (Which path do we take?)

Let's say the Grandparent **(Class A)** has a method called **start().**

- Both **Class B** and **Class C** inherit **start().** They might even change (override) how **start()** works to fit their specific needs.
- Now, **Class D** inherits from **both B** and **C.**
- If you create an object of **Class D** and tell it to run **start(),** the compiler gets confused. Should it run **Class B's** version of **start(),** or **Class C's** version?

2. The Duplication Problem (Two copies of the same data)

If Grandparent **A** has a variable, like **powerLevel = 100,** how does it reach **Class D?**

- It gets passed down through **Class B.**
- It also gets passed down through **Class C.**
- As a result, **Class D** might accidentally inherit **two separate copies** of the **powerLevel** variable. If you change the **powerLevel** using the **B** path, the **powerLevel** on the **C** path remains unchanged. This leads to wasted memory and massive bugs because the object's internal state is out of sync with itself.

Because of this complexity, different programming languages handle it differently:

- **C++:** It allows this structure, but forces you to use a special keyword called **"Virtual Inheritance."** This explicitly tells the compiler, "Only create one copy of the Grandparent, and make the parents share it."
- **Go, JavaScript, and Java** intentionally do **not** support multiple inheritance of classes. Instead, they use concepts like **interfaces** or **mixins** to safely inherit behaviors from multiple sources without inheriting conflicting logic.

Is-A vs Has-A Relationship

1. The "Is-A" Relationship (Inheritance): means that one class is a specialized version of another class. It is implemented using **Inheritance**.

- **The Concept:** When a child class inherently belongs to the same broad category as the parent class and should inherit all of its fundamental behaviors.
- **Tight Coupling:** This creates a very strict, rigid bond between the two classes. If you change the parent, the child is automatically affected.

Examples:

- A Car **is a** Vehicle.
- A Manager **is an** Employee.
- A Circle **is a** Shape.

2. The "Has-A" Relationship (Composition / Aggregation): means that one class contains, or uses another class. It is implemented using **Composition** or **Aggregation**, in which an object of one class is created as a variable within another class.

- **The Concept:** When a class needs the functionality of another class, but it isn't a type of that class. It is about assembling complex objects from smaller, simpler parts.
- **Loose Coupling:** This is much more flexible than inheritance. You can easily swap out the internal components without breaking the main class.

Examples:

- A Car **has an** Engine. (A car uses an engine to drive, but a car is not an engine).
- An Employee **has an** Address.
- A Computer **has a** hard drive.

The Golden Rule of Modern OOP

"Favor Composition over Inheritance." In modern software engineering (like in Go or when using React in JavaScript), developers often prefer the **"Has-A"** relationship. Relying too heavily on **"Is-A"** (deep multilevel inheritance trees) makes code fragile and hard to refactor. Building classes by composing smaller **"Has-A"** parts usually leads to much more modular and testable code.